

PandOSsh — Fase 3

Il Support Level: memoria virtuale, Pager, U-proc e shell
Guida dettagliata riga per riga, funzione per funzione

Materiale di studio per l'interrogazione

A.A. 2025/2026

Indice

1	Introduzione generale alla Fase 3	1
2	Concetti di base: memoria virtuale a paginazione	1
3	phase3/headers/support.h — costanti e variabili globali	2
3.1	Costanti di memoria e Swap Pool	2
3.2	Numeri di syscall e mutex sui device	3
3.3	Variabili globali del Support Level	3
4	phase3/initProc.c — InstantiatorProcess	4
4.1	getSupport — accesso al pool	4
4.2	launchUproc — creazione e avvio di una U-proc	4
4.3	test — l'InstantiatorProcess	5
5	phase3/vmSupport.c — memoria virtuale e Pager	6
5.1	Stato del modulo	6
5.2	Funzioni di utilità (static)	6
5.3	initSwapStructs e initUprocPageTable	7
5.4	flashOperation — I/O sul backing store	8
5.5	pager — il Pager, cuore della memoria virtuale	8
5.6	TLB-Refill vs Pager: la distinzione più chiesta all'esame	10
5.7	Perché installare la traduzione direttamente nel TLB (markPagePresent)	10
6	phase3/sysSupport.c — syscall del Support Level	11
6.1	supTerminate — terminazione ordinata di una U-proc	11
6.2	SYS4 — writeTerminal	11
6.3	SYS5 — readTerminal	12
6.4	SYS6 — doExecute	13
6.5	supSyscallHandler — dispatcher delle syscall	13
6.6	generalExceptionHandler — il punto di ingresso dal Nucleus	13
7	Il boot delle U-proc: crtsi.S e libuser.h	14
7.1	testers/crtsi.S — lo startup in assembly	14
7.2	testers/h/libuser.h — la mini libreria utente	15
8	La shell interattiva (testers/shell.c)	15

9	I programmi U-proc lanciabili dalla shell (testers/)	16
9.1	echo.c — ASID 2	16
9.2	fibEleven.c — ASID 3	17
9.3	uname.c — ASID 4	17
9.4	sl.c — ASID 5	17
9.5	calc.c — ASID 6	17
10	La pipeline di compilazione: da .c a device flash	19
10.1	Il target phase3 in CMakeLists.txt	19
10.2	testers/Makefile — compilazione delle U-proc	20
10.3	phase3_config_machine.json — la configurazione dell'emulatore	20
11	Mappa riassuntiva della memoria (Fase 3)	21

1 Introduzione generale alla Fase 3

La **Phase 3** (Support Level, il quarto livello dello strato complessivo del sistema) è costruita *esclusivamente* usando le 10 syscall del Nucleus esposte dalla Fase 2 (**SYSCALL(...)**): non tocca mai direttamente le strutture dati interne del kernel (PCB, ASL, ready queue). Il suo compito è realizzare due concetti che il Nucleus non conosce affatto:

1. la **memoria virtuale a paginazione** per i processi utente (*U-proc*), con relativo gestore dei page fault (il *Pager*);
2. i **processi utente** veri e propri, che girano in **user-mode** con un proprio spazio di indirizzamento logico (*kuseg*), le loro syscall dedicate (**TERMINATE**, **WRITETERMINAL**, **READTERMINAL**, **EXECUTE**), e — nel caso specifico di questo progetto — una **shell interattiva** che permette di lanciare altri programmi a runtime.

Il codice è distribuito su tre moduli C più uno assembly, con responsabilità nettamente separate:

File	Ruolo
initProc.c	L' <i>InstantiatorProcess</i> (test): inizializza le strutture del Support Level, avvia la shell, atten-
vmSupport.c	Lo <i>Swap Pool</i> , il <i>Pager</i> (gestore dei page fault) con rimpiazzo FIFO, la lettura/scrittura sul
sysSupport.c	Il <i>general exception handler</i> , le syscall del Support Level (TERMINATE , WRITETERMINAL , READ
testers/crtsi.S	Il “C runtime startup” delle U-proc: codice assembly minimale eseguito prima di main di og-

A questi si affianca il **uTLB_RefillHandler** (fisicamente collocato in **phase2/exceptions.c**, compilato condizionalmente con **#ifdef SUPPORT_LEVEL**), perché resta comunque responsabilità del Nucleus popolare il Pass Up Vector, anche se il contenuto dell'handler serve strutture dati (la Page Table) definite dalla Fase 3.

2 Concetti di base: memoria virtuale a paginazione

Prima di entrare nei singoli file, è indispensabile avere chiari alcuni concetti generali di teoria dei sistemi operativi che l'implementazione concreta (Sezioni 3–5) dà per scontati.

Indirizzo logico vs fisico

Una U-proc vede solo **indirizzi logici** (o *virtuali*), nel proprio spazio **kuseg** a partire da **0x80000000**: non conosce e non può indirizzare direttamente la RAM reale. Ogni accesso a memoria viene tradotto dall'hardware in un **indirizzo fisico** (un frame reale di RAM) tramite la **Page Table**. Questo dà a ciascuna U-proc l'illusione di avere la macchina (e uno spazio di indirizzamento che parte sempre da **0x80000000**) tutta per sé, isolandola dalle altre.

Pagina e frame

Lo spazio logico è diviso in blocchi di dimensione fissa (**pagine**, qui `PAGESIZE = 4096` byte); la RAM fisica è divisa negli stessi blocchi (**frame**). Tradurre un indirizzo significa trovare *in quale frame fisico* risiede la pagina logica richiesta.

Page Table

Tabella (una per processo) che mappa ogni pagina logica al frame fisico che la contiene, più alcuni bit di controllo: qui *V* (*valid*, la pagina è presente in RAM) e *D* (*dirty*, la pagina è scrivibile/è stata modificata). In PandOSsh ogni U-proc ha una Page Table **privata** di 32 entry (`sup_privatePgTbl`, dentro la sua `support_t`): 31 pagine per `.text/.data`, 1 per lo stack.

TLB (Translation Lookaside Buffer)

Consultare la Page Table in memoria *a ogni singolo accesso* sarebbe troppo lento: il TLB è una piccola **cache hardware** (qui 16 entry, si veda `"tlb-size": 16` nei file di configurazione) delle traduzioni usate di recente. Se la traduzione richiesta non è nel TLB si parla di **TLB miss**: può essere risolto rapidamente (*TLB-Refill*, se la pagina è comunque presente in Page Table) o richiedere l'intervento del Pager (se la pagina non è affatto in memoria).

ASID (Address Space Identifier)

Un piccolo identificatore (`entry_hi`, $\in [1, 8]$ in questo progetto) che “etichetta” ogni traduzione nel TLB con il processo a cui appartiene. Senza ASID, il TLB andrebbe **completamente invalidato** a ogni cambio di processo (altrimenti un processo potrebbe leggere/scrivere per errore le pagine fisiche di un altro tramite una traduzione stantia); con l'ASID, più traduzioni di processi diversi possono coesistere nel TLB, e l'hardware usa solo quelle che corrispondono al processo in esecuzione.

Page fault

L'eccezione generata quando si accede a una pagina *non presente* in Page Table ($V=0$). È il segnale che fa scattare il **Pager** (Sezione 5.5), il gestore che dovrà trovarla e caricarla.

Paginazione a domanda (*demand paging*)

Strategia per cui nessuna pagina viene caricata in RAM finché non è effettivamente richiesta: ogni U-proc parte con Page Table interamente invalida (Sezione 5.3) e le sue pagine vengono popolate una a una, al primo page fault su ciascuna. Evita di caricare in memoria codice/dati mai usati.

Backing store e swapping

Poiché la RAM fisica non basta a tenere in memoria *tutte* le pagine di *tutte* le U-proc contemporaneamente, serve un supporto di memorizzazione più lento ma più capiente (qui, un device **flash** per U-proc) su cui tenere le pagine non attualmente in RAM: è il **backing store**. Quando la RAM disponibile per la memoria virtuale (lo **Swap Pool**, Sezione 5) è piena, occorre **sfrattare** (*swap out*) una pagina già presente per farne spazio a quella richiesta, riscrivendola sul suo backing store se necessario.

Questi concetti sono **teoria generale dei sistemi operativi**, validi ben oltre PandOSsh; le sezioni successive mostrano *come* questo progetto specifico li implementa (Swap Pool a 16 frame, algoritmo FIFO, device flash come backing store, Page Table a 32 entry per processo).

3 phase3/headers/support.h — costanti e variabili globali

3.1 Costanti di memoria e Swap Pool

```
1 #define SWAP_POOL_START (RAMSTART + (OSFRAMES * PAGESIZE))
2 #define SWAP_POOL_SIZE POOLSIZE
3
4 #define KUSEG_VPN_START 0x80000
5 #define KUSEG_STACK_VPN 0xBFFFF
6 #define UPROC_STACKPAGE 31
7 #define UPROC_TEXTPAGES 31
8
9 #define SWAP_FRAME_FREE (-1)
10
```

```

11 #define UPROC_STATUS (MSTATUS_MPIE_MASK | MSTATUS_MPP_U)
12 #define SUPPORT_STATUS (MSTATUS_MIE_MASK | MSTATUS_MPIE_MASK | MSTATUS_MPP_M)

```

- **SWAP_POOL_START**: la RAM fisica inizia a **RAMSTART** (0x20000000); i primi **OSFRAMES** (32) frame sono riservati al Nucleus (codice, dati, stack degli handler). Lo **Swap Pool** inizia subito dopo, a $0x20000000 + 32 \times 4096 = 0x20020000$.
- **SWAP_POOL_SIZE = POOLSIZE = UPROC_MAX × 2 = 16**: il numero di frame fisici disponibili per lo Swap Pool è il doppio del numero massimo di U-proc (**UPROC_MAX** = 8), una scelta standard di Pandos che garantisce che ogni U-proc possa avere in memoria almeno due pagine contemporaneamente senza causare sfratti immediati reciproci.
- **KUSEG_VPN_START = 0x80000**: il Virtual Page Number della prima pagina logica di una U-proc (corrisponde all'indirizzo virtuale $0x80000000 = \text{KUSEG}$, spostato a destra di **VPNSHIFT**=12 bit).
- **KUSEG_STACK_VPN = 0xBFFFF**: il VPN della pagina di stack (indirizzo virtuale **0xBFFFF000**, appena sotto **USERSTACKTOP = 0xC0000000**).
- **UPROC_STACKPAGE = 31**: nella Page Table privata di 32 entry (indici 0–31), la pagina di stack occupa sempre l'**ultimo** indice; **UPROC_TEXTPAGES = 31**: le prime 31 entry (indici 0–30) sono riservate a **.text/.data**.
- **SWAP_FRAME_FREE = -1**: valore sentinella che, nel campo **sw_asid** di una entry **swap_t**, indica che quel frame dello Swap Pool è attualmente libero (nessuna pagina di nessuna U-proc vi risiede).
- **UPROC_STATUS**: lo stato iniziale del processore per una U-proc: **MSTATUS_MPIE_MASK** (interrupt precedenti abilitati, che diventeranno gli interrupt correnti al primo **LDST**) e **MSTATUS_MPP_U** (Machine Previous Privilege = user, cioè la U-proc girerà in **user-mode**).
- **SUPPORT_STATUS**: lo stato usato per i *context* degli handler del Support Level (Pager, general exception handler): kernel-mode (**MPP_M**) con interrupt e PLT abilitati — questi handler, pur facendo parte “concettualmente” del livello utente, girano a un privilegio più alto per poter eseguire le **SYSCALL** del Nucleus.

3.2 Numeri di syscall e mutex sui device

```

1 #define SUP_TERMINATE      2
2 #define SUP_WRITEPRINTER  3
3 #define SUP_WRITETERMINAL 4
4 #define SUP_READTERMINAL  5
5 #define SUP_EXECUTE       6
6
7 #define MAXSTRLEN 128
8
9 #define DEVCNT          (DEVINTNUM * DEVPERINT)
10 #define DEV_MUTEX_TOTAL (DEVCNT + DEVPERINT)
11
12 #define FLASH_MUTEX(dev) (((IL_FLASH - IL_DISK) * DEVPERINT) + (dev))
13 #define PRINTER_MUTEX(dev) (((IL_PRINTER - IL_DISK) * DEVPERINT) + (dev))
14 #define TERMW_MUTEX(dev) (((IL_TERMINAL - IL_DISK) * DEVPERINT) + (dev))
15 #define TERMR_MUTEX(dev) (DEVCNT + (dev))

```

Le syscall del Support Level hanno codice **positivo** (in contrasto con quelle del Nucleus, negative): 2=TERMINATE, 4=WRITETERMINAL, 5=READTERMINAL, 6=EXECUTE (3=WRITEPRINTER è definita ma non utilizzata dai programmi di test forniti). L'array **devMutex** fornisce un semaforo binario *per ogni sotto-device* (48 in totale: 40 device “principali” su 5 linee × 8, più 8 canali di ricezione terminale separati), per serializzare l'accesso quando più U-proc contendono lo stesso device fisico (tipicamente il flash durante il paging, o il terminale 0 condiviso da shell e programmi).

3.3 Variabili globali del Support Level

```

1 extern int swapPoolSem;
2 extern swap_t swapPool[SWAP_POOL_SIZE];
3 extern int masterSemaphore;
4 extern int shellSemaphore;
5 extern int devMutex[DEV_MUTEX_TOTAL];
6 extern support_t supportPool[UPROCMAX];

```

- `swapPoolSem`: semaforo binario (mutex) che protegge l'intera Swap Pool table: garantisce che un solo page fault alla volta possa manipolare i frame fisici e il backing store.
- `swapPool[16]`: la tabella che descrive il contenuto di ciascun frame fisico dello Swap Pool.
- `masterSemaphore`: su cui si blocca l'`InstantiatorProcess` in attesa che la shell termini.
- `shellSemaphore`: su cui si blocca la shell in attesa che la U-proc appena lanciata (via `EXECUTE`) termini.
- `devMutex[48]`: mutex per sotto-device.
- `supportPool[8]`: pool statico di support structure, una per ciascuna delle 8 possibili U-proc (indicizzato per `ASID-1`).

4 phase3/initProc.c — InstantiatorProcess

4.1 getSupport — accesso al pool

```

1 support_t *getSupport(int asid) {
2     return &supportPool[asid - 1];
3 }

```

Funzione banale ma centrale: dato un `ASID` $\in [1, 8]$, restituisce l'indirizzo della sua support structure nel pool statico. Il legame *uno-a-uno e statico* tra `ASID` e slot dell'array elimina qualunque necessità di allocazione dinamica o di una tabella di lookup più complessa.

4.2 launchUproc — creazione e avvio di una U-proc

```

1 void launchUproc(int asid) {
2     support_t *sup = getSupport(asid);
3
4     sup->sup_asid = asid;
5     initUprocPageTable(sup);
6
7     sup->sup_exceptContext[PGFAULTEXCEPT].pc      = (memaddr) pager;
8     sup->sup_exceptContext[PGFAULTEXCEPT].status  = SUPPORT_STATUS;
9     sup->sup_exceptContext[PGFAULTEXCEPT].stackPtr =
10         (memaddr) &sup->sup_stackTLB[499];
11
12     sup->sup_exceptContext[GENERALEXCEPT].pc      = (memaddr) generalExceptionHandler;
13     sup->sup_exceptContext[GENERALEXCEPT].status  = SUPPORT_STATUS;
14     sup->sup_exceptContext[GENERALEXCEPT].stackPtr =
15         (memaddr) &sup->sup_stackGen[499];
16
17     state_t s;
18     for (unsigned int i = 0; i < (STATE_T_SIZE_IN_BYTES / WORDLEN); i++)
19         ((unsigned int *)&s)[i] = 0;
20
21     s.pc_epc      = UPROCSTARTADDR;
22     s.reg_sp      = USERSTACKTOP;
23     s.status      = UPROC_STATUS;
24     s.mie         = MIE_ALL;
25     s.entry_hi    = asid << ASIDSHIFT;
26
27     SYSCALL(CREATEPROCESS, (int)&s, PROCESS_PRIO_LOW, (int)sup);
28 }

```

Questa funzione fa *tutto il necessario* per portare in vita una nuova U-proc, sia essa la shell (ASID 1, lanciata dall'InstantiatorProcess) o un programma qualsiasi lanciato dalla shell tramite EXECUTE (ASID 2–8). Passi:

1. Recupera la support structure dedicata a quell'ASID e vi registra l'ASID stesso.
2. `initUprocPageTable(sup)`: inizializza le 32 entry della Page Table privata (vedi `vmSupport.c` più sotto) tutte come *non presenti*: il primo accesso a qualunque pagina genererà un page fault, che il Pager gestirà caricandola dal backing store.
3. **Configura i due context_t** usati da `passUpOrDie` nel Nucleus per il pass-up delle eccezioni: uno per i page fault (PGFAULTEXCEPT, indice 0), che punta a `pager` (in `vmSupport.c`), e uno per tutte le altre eccezioni (GENERALEXCEPT, indice 1), che punta a `generalExceptionHandler` (in `sysSupport.c`). Ciascuno dei due usa **uno stack separato** interno alla support structure (`sup_stackTLB`, `sup_stackGen`, 500 word ciascuno, puntatore inizializzato all'ultima word valida [499], dato che lo stack cresce verso il basso). **Stack distinti sono fondamentali**: se un page fault avviene mentre si sta ancora gestendo una syscall (e quindi si sta usando lo stack `sup_stackGen`), usare lo stesso stack per il Pager ne corromperebbe il contenuto.
4. **Costruisce lo stato iniziale s** della U-proc, azzerando prima tutta la struttura (ciclo manuale word per word, analogo a `copyState` ma da zero anziché da una sorgente) e poi impostando:
 - `pc_epc = UPROCSTARTADDR (0x800000B0)`: il punto di ingresso *non* è `main` della U-proc, ma il simbolo `__start` definito da `crtsi.S` (vedi più avanti), collocato dal linker esattamente a questo indirizzo (subito dopo l'header `.aout`);
 - `reg_sp = USERSTACKTOP (0xC0000000)`: cima dello stack utente, che coincide con l'indirizzo appena sopra la pagina di stack mappata all'indice 31 della Page Table;
 - `status = UPROC_STATUS`: user-mode, interrupt abilitati;
 - `mie = MIE_ALL`: tutte le sorgenti di interrupt abilitate a livello di maschera;
 - `entry_hi = asid « ASIDSHIFT`: registra l'ASID univoco nel campo alto del registro `EntryHI`, così che ogni traduzione TLB generata da questa U-proc sia automaticamente taggata con il suo ASID (permettendo al TLB di distinguere le pagine di U-proc diverse senza doverlo invalidare interamente a ogni cambio di processo).
5. **Materializza il processo tramite il Nucleus**: `SYSCALL(CREATEPROCESS, ...)` (`SYS1`, la *unica* syscall del Nucleus usata per creare processi, sia kernel sia utente) con lo stato appena costruito, priorità bassa, e il puntatore alla support structure (che rende il PCB creato una vera U-proc agli occhi del Nucleus: da questo momento, ogni eccezione che il Nucleus non sa gestire per questo processo verrà passata su tramite `passUpOrDie` ai due context configurati sopra).

4.3 test — l'InstantiatorProcess

```
1 void test(void) {
2     initSwapStructs();
3
4     masterSemaphore = 0;
5     shellSemaphore = 0;
6     for (int i = 0; i < DEV_MUTEX_TOTAL; i++)
7         devMutex[i] = 1;
8
9     launchUproc(1);
10
11     SYSCALL(PASSEREN, (int)&masterSemaphore, 0, 0);
12
13     SYSCALL(TERMPROCESS, 0, 0, 0);
14 }
```

`test` è la funzione con cui il Nucleus fa partire il *primo* processo del sistema (il suo indirizzo è impostato come `pc_epc` del PCB creato in `initial.c::main`). Nella build `phase3` questa funzione

(definita qui, non nel `p2test.c` usato dalla Fase 2 pura) è l'**InstantiatorProcess**, e gira interamente in **kernel-mode** (non ha una support struct propria: è un processo puro Fase 2). Sequenza:

1. `initSwapStructs()`: inizializza lo Swap Pool e il suo semaforo di mutua esclusione (dettagli in `vmSupport.c`).
2. Inizializza i semafori di sincronizzazione del Support Level a 0 (`masterSemaphore`, `shellSemaphore`) e tutti i 48 mutex dei device a 1 (semafori binari liberi).
3. **Avvia una sola U-proc: la shell**, con `ASID=1` (`launchUproc(1)`). È questa la scelta architetturale distintiva di PandOSsh rispetto allo schema “classico” di Pandos, che normalmente lancerebbe direttamente tutte le U-proc di test in sequenza: qui, invece, l'unico processo lanciato dal kernel è una shell interattiva, e sono le *syscall EXECUTE* della shell stessa (invocate a runtime in base a cosa l'utente digita) a creare, una alla volta, le altre U-proc.
4. **Si blocca su masterSemaphore** con una `PASSEREN` (P, syscall del Nucleus): l'InstantiatorProcess resta bloccato finché la shell (`ASID 1`) non esegue una V su questo stesso semaforo — cosa che accade in `supTerminate` quando la shell termina (comando `exit`).
5. **Sbloccato**, esegue `TERMPROCESS` su se stesso (`a1=0`): essendo l'InstantiatorProcess l'ultimo processo rimasto vivo a questo punto (tutte le U-proc sono già terminate, essendo la shell l'unica lanciata direttamente e le sue figlie terminate prima di lei), questa terminazione porta `processCount` a 0, e lo scheduler (al successivo giro) esegue `HALT()`, fermando la macchina virtuale.

5 phase3/vmSupport.c — memoria virtuale e Pager

5.1 Stato del modulo

```
1 int swapPoolSem;
2 swap_t swapPool[SWAP_POOL_SIZE];
3 static int fifoNext = 0;
```

`fifoNext` è l'indice (privato del modulo) usato dall'algoritmo di rimpiazzo pagine: parte da 0 e avanza ciclicamente su tutti i 16 frame dello Swap Pool, realizzando un semplice **round-robin** (qui usato come approssimazione di FIFO: il frame scelto è sempre il successivo nell'ordine circolare, indipendentemente da quando è stato effettivamente occupato — è comunque l'algoritmo che la specifica di Pandos ammette esplicitamente per la sua semplicità e prevedibilità).

5.2 Funzioni di utilità (static)

```
1 static inline void interruptsOff(void) {
2     setSTATUS(getSTATUS() & ~MSTATUS_MIE_MASK);
3 }
4 static inline void interruptsOn(void) {
5     setSTATUS(getSTATUS() | MSTATUS_MIE_MASK);
6 }
```

Due wrapper minimi per disabilitare/riabilitare il bit MIE dello `status` corrente: usati per rendere *atomiche* le operazioni combinate di aggiornamento Page Table + TLB (vedi sotto).

```
1 static int vpnToIndex(unsigned int entryHI) {
2     unsigned int vpn = (entryHI >> VPNSHIFT) & 0xFFFFF;
3     if (vpn == KUSEG_STACK_VPN) return UPROC_STACKPAGE;
4     if (vpn >= KUSEG_VPN_START && vpn < KUSEG_VPN_START + UPROC_TEXTPAGES)
5         return (int)(vpn - KUSEG_VPN_START);
6     return -1;
7 }
```

`vpnToIndex`: dato il valore del registro `EntryHI` salvato al momento del page fault, estrae il VPN (bit alti, shiftati di `VPNSHIFT=12`) e lo traduce nell'indice `[0..31]` della Page Table privata: caso speciale per la pagina di stack (VPN `0xBFFFF` → indice 31), altrimenti pagine `.text/.data` (VPN

in [0x80000, 0x8001E] → indici 0–30). Se il VPN richiesto non rientra in nessuno dei due intervalli, l'indirizzo è **fuori dallo spazio logico** della U-proc: si ritorna -1, che il chiamante (**pager**) interpreta come errore fatale (program trap → terminazione).

```
1 static inline memaddr frameAddr(int i) {
2     return (memaddr)(SWAP_POOL_START + i * PAGESIZE);
3 }
```

Calcola l'indirizzo fisico del frame *i*-esimo dello Swap Pool (banale aritmetica: base più *i* pagine).

```
1 static void markPageNotValid(pteEntry_t *pte) {
2     interruptsOff();
3     pte->pte_entryLO &= ~VALIDON;
4     TLBCLR();
5     interruptsOn();
6 }
```

markPageNotValid: invalida un'entry di Page Table (azzerà il bit **V**, **VALIDON**) e cancella tutto il TLB (**TLBCLR**) in modo **atomico** (interrupt disabilitati durante l'operazione). Usata quando si sfratta una pagina vittima dallo Swap Pool: da questo momento qualunque accesso a quella pagina, da parte della sua U-proc, genererà un nuovo page fault.

```
1 static void markPagePresent(pteEntry_t *pte, memaddr phys) {
2     interruptsOff();
3     pte->pte_entryLO = phys | DIRTYON | VALIDON;
4     TLBCLR();
5     setENTRYHI(pte->pte_entryHI);
6     setENTRYLO(pte->pte_entryLO);
7     TLBWR();
8     interruptsOn();
9 }
```

markPagePresent: rende presente un'entry di Page Table, impostando il frame fisico (**phys**) e i bit **D** (dirty, scrivibile) e **V** (valida). **Scelta implementativa degna di nota**: oltre a cancellare il TLB con **TLBCLR** (che elimina eventuali entry stantie relative al vecchio contenuto del frame), la nuova traduzione viene scritta **direttamente** nel TLB (**setENTRYHI**/**setENTRYLO**/**TLBWR**), invece di limitarsi a invalidare e aspettare che sia un futuro evento di TLB-Refill a ripopolarla. La specifica di Pandos ammette entrambe le strategie (cancellare tutto il TLB, oppure individuare/riscrivere la singola entry); questa implementazione adotta una variante della seconda, restando quindi pienamente conforme. La motivazione pratica di questa scelta (rispetto alla più semplice “solo **TLBCLR**”) è discussa più sotto, dopo aver introdotto il Pager: era stato osservato un *page-fault loop* in cui l'evento di TLB-Refill non veniva sempre rigenerato correttamente subito dopo un fault gestito dal Pager, lasciando la U-proc bloccata a ripetere indefinitamente lo stesso fault.

5.3 initSwapStructs e initUprocPageTable

```
1 void initSwapStructs(void) {
2     swapPoolSem = 1;
3     fifoNext = 0;
4     for (int i = 0; i < SWAP_POOL_SIZE; i++)
5         swapPool[i].sw_asid = SWAP_FRAME_FREE;
6 }
```

Inizializza il semaforo dello Swap Pool a 1 (mutex libero), azzerà l'indice FIFO, e marca tutti i 16 frame come liberi. Chiamata una sola volta dall'InstantiatorProcess (**test**).

```
1 void initUprocPageTable(support_t *sup) {
2     int asid = sup->sup_asid;
3     for (int i = 0; i < UPROC_TEXTPAGES; i++) {
4         sup->sup_privatePgTbl[i].pte_entryHI =
5             ((KUSEG_VPN_START + i) << VPNSHIFT) | (asid << ASIDSHIFT);
6         sup->sup_privatePgTbl[i].pte_entryLO = DIRTYON;
7     }
8     sup->sup_privatePgTbl[UPROC_STACKPAGE].pte_entryHI =
9         (KUSEG_STACK_VPN << VPNSHIFT) | (asid << ASIDSHIFT);
10    sup->sup_privatePgTbl[UPROC_STACKPAGE].pte_entryLO = DIRTYON;
11 }
```


Per ciascuna delle 32 entry della Page Table privata, imposta `pte_entryHI` con il VPN corretto (progressivo da `KUSEG_VPN_START` per le prime 31, fisso a `KUSEG_STACK_VPN` per l'ultima) taggato con l'ASID della U-proc, e `pte_entryLO` al solo bit `DIRTYON` (scrivibile) **senza** `VALIDON`: ogni pagina parte quindi **non presente** in memoria. Il primo accesso a *qualunque* pagina della U-proc genererà perciò un page fault gestito dal Pager, che la caricherà a domanda dal backing store (paginazione *lazy*, tipica dei sistemi a domanda di pagina).

5.4 flashOperation — I/O sul backing store

```

1 int flashOperation(int asid, int blockNo, memaddr frameAddr_, int op) {
2     int devNo = asid - 1;
3     dtpreg_t *flash = (dtpreg_t *) DEV_REG_ADDR(IL_FLASH, devNo);
4     int mutex = FLASH_MUTEX(devNo);
5     unsigned int status;
6
7     SYSCALL(PASSEREN, (int)&devMutex[mutex], 0, 0);
8
9     flash->data0 = frameAddr_;
10    unsigned int command = ((unsigned int)blockNo << 8) | op;
11    status = SYSCALL(DOIO, (int)&flash->command, (int)command, 0);
12
13    SYSCALL(VERHOGEN, (int)&devMutex[mutex], 0, 0);
14    return (int)status;
15 }
```

Il **backing store** di ciascuna U-proc è un device flash dedicato: la U-proc con `ASID=i` usa il device flash numero `i-1` (`devNo = asid - 1`), secondo la mappatura fissata anche nel file di configurazione della macchina (`phase3_config_machine.json`).

1. Acquisisce il mutex dedicato a quel device flash (`PASSEREN`), per evitare che due page fault concorrenti (di U-proc diverse, o anche della stessa) tentino di usare lo stesso device contemporaneamente.
2. Imposta `flash->data0` all'indirizzo `frameAddr_`: è il meccanismo **DMA** (*Direct Memory Access*, trasferimento dati che il device esegue direttamente da/verso la RAM, senza passare attraverso il processore) con cui il device sa *da/verso dove* trasferire i dati (il frame fisico coinvolto).
3. Compone il comando: il numero di blocco (`blockNo`, che corrisponde alla pagina logica, 0–31) va nei byte alti, l'opcode (`FLASHREAD` o `FLASHWRITE`) nel byte basso, ed esegue `DOIO` (la syscall del Nucleus) che blocca il chiamante finché l'operazione non è completata dall'interrupt.
4. Rilascia il mutex e restituisce lo status ricevuto.

La mutua esclusione per-device è **deliberatamente separata** da `swapPoolSem`: quest'ultimo protegge la *struttura dati* condivisa (la tabella Swap Pool), mentre `devMutex` protegge l'*hardware* fisico condiviso; tenerli distinti evita di serializzare inutilmente operazioni su flash *diversi* solo perché entrambe capitate durante un page fault (in pratica, dato che c'è comunque un solo `swapPoolSem` globale, un solo page fault alla volta è comunque in corso; la separazione resta comunque corretta e più chiara concettualmente).

5.5 pager — il Pager, cuore della memoria virtuale

```

1 void pager(void) {
2     support_t *sup = (support_t *) SYSCALL(GETSUPPORTPTR, 0, 0, 0);
3     state_t *exState = &sup->sup_exceptState[PGFAULTEXCEPT];
4
5     unsigned int excCode = exState->cause & CAUSE_EXCCODE_MASK;
6
7     if (excCode == EXC_MOD) {
8         supTerminate(sup->sup_asid);
9         return;
10    }
11 }
```

```

12  SYSCALL(PASSEREN, (int)&swapPoolSem, 0, 0);
13
14  int p = vpnToIndex(exState->entry_hi);
15  if (p < 0) {
16      SYSCALL(VERHOGEN, (int)&swapPoolSem, 0, 0);
17      supTerminate(sup->sup_asid);
18      return;
19  }
20
21  int i = fifoNext;
22  fifoNext = (fifoNext + 1) % SWAP_POOL_SIZE;
23  memaddr fa = frameAddr(i);
24
25  if (swapPool[i].sw_asid != SWAP_FRAME_FREE) {
26      int victimAsid = swapPool[i].sw_asid;
27      int victimPage = swapPool[i].sw_pageNo;
28      pteEntry_t *victimPte = swapPool[i].sw_pte;
29
30      markPageNotValid(victimPte);
31
32      int st = flashOperation(victimAsid, victimPage, fa, FLASHWRITE);
33      if (st != READY) {
34          SYSCALL(VERHOGEN, (int)&swapPoolSem, 0, 0);
35          supTerminate(sup->sup_asid);
36          return;
37      }
38  }
39
40  int st = flashOperation(sup->sup_asid, p, fa, FLASHREAD);
41  if (st != READY) {
42      SYSCALL(VERHOGEN, (int)&swapPoolSem, 0, 0);
43      supTerminate(sup->sup_asid);
44      return;
45  }
46
47  swapPool[i].sw_asid = sup->sup_asid;
48  swapPool[i].sw_pageNo = p;
49  swapPool[i].sw_pte = &sup->sup_privatePgTbl[p];
50
51  markPagePresent(&sup->sup_privatePgTbl[p], fa);
52
53  SYSCALL(VERHOGEN, (int)&swapPoolSem, 0, 0);
54  LDST(exState);
55 }

```

Il Pager è il gestore di eccezione a cui il Nucleus salta (tramite `passUpOrDie(PGFAULTEXCEPT)` → `LDCXT`) ogni volta che una U-proc genera un page fault vero (pagina non presente in Page Table). Gira in kernel-mode sullo stack `sup_stackTLB`. Analisi passo-passo:

1. **Recupero del contesto:** `GETSUPPORTPTR` (SYS8 del Nucleus) restituisce la support structure del processo che ha causato l'eccezione (il Nucleus, tramite `passUpOrDie`, aveva già copiato lo stato al momento del fault in `sup->sup_exceptState[PGFAULTEXCEPT]` *prima* di saltare qui: quello è `exState`).
2. **Caso EXC_MOD** (TLB-Modification: tentativo di scrivere su una pagina segnata come read-only): in PandOSsh tutte le pagine utente sono marcate scrivibili (`DIRTYON`), quindi questa condizione non dovrebbe mai verificarsi in condizioni normali; se capitasse (bug applicativo o istruzione malformata) viene trattata come errore fatale → `supTerminate`.
3. **Acquisizione del mutex `swapPoolSem`:** da qui in poi, un solo page fault alla volta può manipolare la Swap Pool table.
4. **Traduzione VPN→indice** con `vpnToIndex`: se l'indirizzo è fuori dallo spazio logico valido, si rilascia il mutex e si termina la U-proc (program trap).
5. **Scelta del frame vittima (algoritmo FIFO):** si prende l'indice corrente `fifoNext` e lo si fa avanzare ciclicamente, indipendentemente dal fatto che il frame scelto sia libero o occupato.
6. **Sfratto (se il frame era occupato):** se `swapPool[i]` contiene già una pagina di qualche U-proc (`sw_asid != SWAP_FRAME_FREE`), quella pagina va *salvata* prima di essere sovrascritta:

- si invalida immediatamente la sua PTE (e il TLB) con `markPageNotValid`, così che nessuno possa più accedervi “credendola” ancora presente mentre viene scritta sul disco;
 - si scrive il contenuto del frame sul backing store *della vittima* (`flashOperation(victimAsid, victimPage, fa, FLASHWRITE)`); se l'operazione fallisce (`status != READY`), si rilascia il mutex e si termina la U-proc *corrente* (quella che ha causato il fault, non la vittima) come errore di I/O fatale.
7. **Caricamento della pagina richiesta:** si legge dal backing store *della U-proc corrente* il blocco `p` nel frame appena liberato (`flashOperation(sup->sup_asid, p, fa, FLASHREAD)`); stesso trattamento in caso di errore.
 8. **Aggiornamento della Swap Pool table** per il frame `i`: ora contiene la pagina `p` della U-proc corrente, con un puntatore diretto alla sua PTE (utile per un futuro, rapido sfratto).
 9. **Aggiornamento atomico di Page Table + TLB** con `markPagePresent`: la PTE della U-proc corrente per la pagina `p` viene marcata presente, puntando al frame fisico appena caricato.
 10. **Rilascio del mutex e ripresa della U-proc** (`LDST(exState)`): l'istruzione che aveva causato il fault verrà rieseguita da capo, questa volta trovando la traduzione valida.

5.6 TLB-Refill vs Pager: la distinzione più chiesta all'esame

	TLB-Refill (uTLB_RefillHandler)	Pager (pager)
Dove	<code>phase2/exceptions.c</code> , guardato da <code>#ifdef SUPPORT_LEVEL</code>	<code>phase3/vmSupport.c</code>
Quando scatta	la traduzione è <i>già presente</i> in Page Table (<code>V=1</code>) ma non è nel TLB (cache)	la pagina è <i>assente</i> in Page Table (<code>V=0</code>): non è mai stata caricata o è stata sfrattata
Costo	rapidissimo: solo una copia PTE→TLB (<code>setENTRYHI/setENTRYLO/TLBWR</code>)	costoso: può richiedere di scrivere una pagina vittima su flash e di leggerne una nuova (due operazioni di I/O bloccanti)
Tocca il disco	mai	sì, se necessario (sfratto + lettura)
Registrato come	<code>tlb_refill_handler</code> nel Pass Up Vector (percorso hardware dedicato)	raggiunto via <code>passUpOrDie(PGFAULTEXCEPT)</code> → LDCXT al context configurato in <code>launchUproc</code>

Il TLB è una *cache* delle traduzioni contenute nella Page Table: può contenere solo poche entry alla volta, quindi anche una pagina *presente* in memoria può generare un evento di TLB-Refill se la sua traduzione non è (più) nella cache. Solo quando la pagina non è affatto in memoria (bit `V=0`) si tratta di un vero page fault, di competenza del Pager.

5.7 Perché installare la traduzione direttamente nel TLB (markPagePresent)

La specifica di Pandos ammette due strategie equivalenti per aggiornare il TLB dopo che il Pager ha reso presente una pagina: (a) limitarsi a `TLBCLR` e lasciare che sia un successivo evento di TLB-Refill a ripopolare l'entry quando servirà; oppure (b) individuare/scrivere direttamente la entry nel TLB. Questa implementazione ha scelto (una variante di) la strategia (b) *dopo aver diagnosticato un problema concreto*: in alcune situazioni l'evento di TLB-Refill non veniva rigenerato in modo affidabile subito dopo l'intervento del Pager, e la U-proc restava intrappolata in un **page-fault loop** (l'accesso che riprendeva dopo il fault ritrovava ancora una traduzione mancante nel TLB, rigenerando lo stesso

fault all'infinito). Scrivendo la traduzione direttamente nel TLB dentro `markPagePresent`, l'accesso che riprende subito dopo (`LDST(exState)`) trova già la entry pronta, eliminando il problema.

6 phase3/sysSupport.c — syscall del Support Level

6.1 supTerminate — terminazione ordinata di una U-proc

```

1 void supTerminate(int asid) {
2     SYSCALL(PASSEREN, (int)&swapPoolSem, 0, 0);
3     for (int i = 0; i < SWAP_POOL_SIZE; i++)
4         if (swapPool[i].sw_asid == asid)
5             swapPool[i].sw_asid = SWAP_FRAME_FREE;
6     SYSCALL(VERHOGEN, (int)&swapPoolSem, 0, 0);
7
8     if (asid == 1)
9         SYSCALL(VERHOGEN, (int)&masterSemaphore, 0, 0);
10    else
11        SYSCALL(VERHOGEN, (int)&shellSemaphore, 0, 0);
12
13    SYSCALL(TERMPROCESS, 0, 0, 0);
14 }

```

Questa è la funzione *unica* tramite cui **qualsunque** U-proc smette di esistere (che sia una terminazione volontaria via `TERMINATE`, un errore fatale rilevato dal Pager, o un program trap gestito dal general exception handler): mai una U-proc chiama direttamente `SYSCALL(TERMPROCESS, ...)` senza passare da qui.

1. **Libera i frame dello Swap Pool** occupati da questa U-proc (sotto `swapPoolSem`): scorre tutta la tabella e marca come liberi (`SWAP_FRAME_FREE`) tutti i frame il cui `sw_asid` coincide con l'ASID che sta terminando. È un passo essenziale: senza di esso, un futuro sfratto potrebbe tentare di scrivere sul backing store di una U-proc ormai terminata (e magari già sostituita da una nuova U-proc con lo stesso ASID, dato che gli ASID vengono riusati), corrompendone i dati.
2. **Sblocca l'attente giusto** in base all'ASID: se la U-proc che termina è la **shell** (`ASID=1`), fa una V su `masterSemaphore` (sbloccando l'InstantiatorProcess, che porterà alla `HALT` finale); altrimenti (una qualunque U-proc *figlia* lanciata dalla shell tramite `EXECUTE`) fa una V su `shellSemaphore` (sbloccando la shell, che tornerà al prompt). Questa distinzione realizza esplicitamente la **catena di attesa** a due livelli: `figlia` → `shell` → `InstantiatorProcess`.
3. **Terminazione effettiva** tramite il Nucleus: `SYSCALL(TERMPROCESS, 0, 0, 0)` (`a1=0`: termina se stesso). Da questo punto il PCB della U-proc viene distrutto dal Nucleus (Fase 2); la funzione non ritorna mai.

6.2 SYS4 — writeTerminal

```

1 static int writeTerminal(support_t *sup, char *virtAddr, int len) {
2     if ((memaddr)virtAddr < KUSEG || (memaddr)virtAddr >= USERSTACKTOP ||
3         len < 0 || len > MAXSTRLEN) {
4         supTerminate(sup->sup_asid);
5     }
6
7     termreg_t *term = (termreg_t *) DEV_REG_ADDR(IL_TERMINAL, 0);
8     int mutex = TERMW_MUTEX(0);
9
10    SYSCALL(PASSEREN, (int)&devMutex[mutex], 0, 0);
11
12    int i;
13    for (i = 0; i < len; i++) {
14        unsigned int cmd =
15            (((unsigned int)(unsigned char)virtAddr[i]) << 8) | TRANSMITCHAR;
16        unsigned int status =
17            SYSCALL(DOIO, (int)&term->transm_command, (int)cmd, 0);
18        if ((status & 0xFF) != OKCHARTRANS) {

```

```

19     SYSCALL(VERHOGEN, (int)&devMutex[mutex], 0, 0);
20     return -(int)(status & 0xFF);
21 }
22 }
23
24 SYSCALL(VERHOGEN, (int)&devMutex[mutex], 0, 0);
25 return i;
26 }

```

Scrive `len` caratteri dal buffer `virtAddr` (indirizzo *logico*, nello spazio della U-proc chiamante) sul terminale 0 (condiviso da tutte le U-proc e dalla shell).

1. **Validazione dell'indirizzo:** deve cadere dentro lo spazio logico `[KUSEG, USERSTACKTOP)` della U-proc, e la lunghezza deve essere non negativa e non superiore a `MAXSTRLEN` (128). Se la richiesta è malformata, si applica la politica “input non valido → program trap”: la U-proc viene terminata immediatamente (`supTerminate`, che non ritorna).
2. **Mutua esclusione:** acquisisce il mutex dedicato alla *scrittura* sul terminale 0 (`TERMR_MUTEX(0)`) — distinto dal mutex di lettura, perché TX e RX sono canali indipendenti e serializzarli insieme impedirebbe inutilmente scritture e letture concorrenti da parte di U-proc diverse.
3. **Ciclo di trasmissione:** per ciascun carattere, compone il comando (carattere nel byte alto, opcode `TRANSMITCHAR` nel byte basso) e lo emette con `DOIO` (bloccando finché il carattere non è stato trasmesso). Se lo status ricevuto non è `OKCHARTRANS` (5, successo), l'operazione si interrompe restituendo lo status come valore *negativo* (convenzione: successo → numero di caratteri scritti, errore → codice di errore negato).
4. Rilascia il mutex e ritorna il numero di caratteri effettivamente scritti (o l'errore).

6.3 SYS5 — readTerminal

```

1 static int readTerminal(support_t *sup, char *virtAddr) {
2     if ((memaddr)virtAddr < KUSEG || (memaddr)virtAddr >= USERSTACKTOP) {
3         supTerminate(sup->sup_asid);
4     }
5
6     termreg_t *term = (termreg_t *) DEV_REG_ADDR(IL_TERMINAL, 0);
7     int mutex = TERMR_MUTEX(0);
8
9     SYSCALL(PASSEREN, (int)&devMutex[mutex], 0, 0);
10
11     int count = 0;
12     int done = 0;
13     while (!done) {
14         unsigned int status =
15             SYSCALL(DOIO, (int)&term->recv_command, (int)RECEIVECHAR, 0);
16         if ((status & 0xFF) != CHARRECV) {
17             SYSCALL(VERHOGEN, (int)&devMutex[mutex], 0, 0);
18             return -(int)(status & 0xFF);
19         }
20         char c = (char)((status >> 8) & 0xFF);
21         virtAddr[count++] = c;
22         if (c == '\n') done = 1;
23     }
24
25     SYSCALL(VERHOGEN, (int)&devMutex[mutex], 0, 0);
26     return count;
27 }

```

Legge caratteri dal terminale 0 nel buffer `virtAddr` **finché non incontra un `'\n'`** (un vero “leggi una riga”). Stessa validazione dell'indirizzo di `writeTerminal` (qui senza controllo su una lunghezza massima, dato che non è un parametro della syscall). Acquisisce il mutex *di lettura* (`TERMR_MUTEX(0)`), indipendente da quello di scrittura). Ciclo: ad ogni iterazione emette un comando `RECEIVECHAR` con `DOIO` (bloccando finché non arriva un carattere), estrae il carattere ricevuto dal byte alto dello status, lo scrive nel buffer e incrementa il contatore; il ciclo termina quando il carattere letto è `'\n'`. Se in un qualunque momento lo status non è `CHARRECV` (5), l'operazione si interrompe restituendo l'errore negato. Al termine restituisce il numero di caratteri letti (incluso il `'\n'` finale).

6.4 SYS6 — doExecute

```
1 static int doExecute(int asid) {
2     if (asid < 1 || asid > UPROCMAX)
3         return -1;
4     launchUproc(asid);
5     SYSCALL(PASSEREN, (int)&shellSemaphore, 0, 0);
6     return 0;
7 }
```

Implementa la syscall con cui la shell lancia un nuovo programma: valida che l'ASID richiesto sia nell'intervallo [1, UPROCMAX] (altrimenti ritorna -1 senza fare nulla — *non* termina il chiamante, a differenza delle altre syscall: è un errore “recuperabile”, gestito dalla shell stessa che stampa un messaggio e richiede un nuovo comando), poi chiama `launchUproc(asid)` (la stessa funzione usata dall'InstantiatorProcess per lanciare la shell) e si **blocca** su `shellSemaphore`: è esattamente questo blocco che rende la shell **sincrona** — un solo programma alla volta, il prompt torna disponibile solo dopo che il programma appena lanciato ha eseguito `supTerminate` (che, per un ASID $\neq 1$, fa la V liberatoria proprio su `shellSemaphore`).

6.5 supSyscallHandler — dispatcher delle syscall

```
1 static void supSyscallHandler(support_t *sup, state_t *state) {
2     int number = (int)state->reg_a0;
3     int result = 0;
4
5     switch (number) {
6         case SUP_TERMINATE:
7             supTerminate(sup->sup_asid);
8             return;
9
10        case SUP_WRITETERMINAL:
11            result = writeTerminal(sup, (char *)state->reg_a1, (int)state->reg_a2);
12            break;
13
14        case SUP_READTERMINAL:
15            result = readTerminal(sup, (char *)state->reg_a1);
16            break;
17
18        case SUP_EXECUTE:
19            result = doExecute((int)state->reg_a1);
20            break;
21
22        default:
23            supTerminate(sup->sup_asid);
24            return;
25    }
26
27    state->reg_a0 = (unsigned int)result;
28    state->pc_epc += WORDLEN;
29    LDST(state);
30 }
```

Legge il numero di syscall da `a0` e smista alle quattro funzioni viste sopra. Le syscall `TERMINATE` *non ritornano mai* qui (chiamano `supTerminate` che a sua volta non ritorna). Le altre tre restituiscono un intero (`result`), che al termine dello `switch` viene scritto in `a0` dello stato salvato; il PC viene avanzato di una word (per non rieseguire la `ecall` originale) e si riprende la U-proc con `LDST`. Qualunque numero di syscall non riconosciuto (fuori dai quattro gestiti) viene trattato come errore fatale: `supTerminate`.

6.6 generalExceptionHandler — il punto di ingresso dal Nucleus

```
1 void generalExceptionHandler(void) {
2     support_t *sup = (support_t *) SYSCALL(GETSUPPORTPTR, 0, 0, 0);
3     state_t *state = &sup->sup_exceptState[GENERALEXCEPT];
4
5     unsigned int excCode = state->cause & CAUSE_EXCCODE_MASK;
```

```

6
7     if (excCode == EXC_ECU) {
8         supSyscallHandler(sup, state);
9     } else {
10        supTerminate(sup->sup_asid);
11    }
12 }

```

Questo è l'indirizzo configurato in `launchUproc` come `sup_exceptContext[GENERALEXCEPT].pc`: è dove il Nucleus salta (tramite `passUpOrDie(GENERALEXCEPT) → LDCXT`) per *qualunque* eccezione non-page-fault generata da una U-proc: syscall positive ($a0 \geq 1$), program trap generici (istruzioni illegali, accessi non allineati, ecc.). Recupera la propria support structure (`GETSUPPORTPTR`) e lo stato salvato in `sup_exceptState[GENERALEXCEPT]`; distingue:

- se l'`ExcCode` è `EXC_ECU` (8, `ecall` da user-mode): è davvero una richiesta di syscall → `supSyscallHandler`;
- altrimenti: è un vero **program trap** (errore di programma) → terminazione immediata della U-proc con `supTerminate`.

Questa è la politica generale della Fase 3: *qualunque* eccezione imprevista o input malformato porta sempre e solo alla terminazione ordinata della U-proc coinvolta, mai a un crash del sistema.

7 Il boot delle U-proc: `crtsi.S` e `libuser.h`

7.1 `testers/crtsi.S` — lo startup in assembly

```

1  #define TEXT_VADDR 0x80000000
2  #define TERMINATE_SYSCALL 2
3
4  .text
5  .align 2
6  .globl __start
7  .type __start, @function
8  .extern main
9  __start:
10     li t0, TEXT_VADDR + (AOUT_HE_GP_VALUE << 2)
11     lw gp, 0(t0)
12
13     addi sp, sp, -16
14     jal main
15     addi sp, sp, 16
16
17     li a0, TERMINATE_SYSCALL
18     ecall
19     /* Not reached! */
20     .end __start
21     .size __start, .-__start

```

Ogni programma utente in `testers/` (shell, echo, fibEleven, uname, sl, calc) viene collegato con questo modulo assembly (insieme a `liburiscv.S` della libreria di sistema): è il “C runtime startup”, l'equivalente per le U-proc di ciò che `crtso.S` è per il kernel. È codice che gira interamente in **user-mode**, eseguito *prima* di `main`.

1. Definisce il simbolo `__start`, che il linker (con lo script `uriscvaout.ldscript`) colloca esattamente all'indirizzo `0x800000B0` (`UPROCSTARTADDR`), cioè subito dopo l'header `.aout` che precede il `.text` vero e proprio (che parte a `0x80000000 = TEXT_VADDR`). È questo, non `main`, il punto in cui `launchUproc` fa partire il PC della U-proc.
2. **Carica il global pointer** (`gp`) leggendo il suo valore dall'header `.aout` (campo `AOUT_HE_GP_VALUE`, posizionato appena dopo `TEXT_VADDR`): senza `gp` correttamente inizializzato, l'accesso a variabili globali/statiche del programma (indirizzate in modo relativo a `gp` nella convenzione RISC-V) fallirebbe.
3. **Alloca un piccolo frame** sullo stack (16 byte, per allineamento) e chiama `main` con `jal` (lo `sp` iniziale — `0xC0000000` — è già stato impostato dal kernel in `launchUproc`, prima che questo codice inizi a girare).

4. **Se main ritorna** (comportamento normale per la maggior parte dei programmi di test, tranne la shell che ha un loop infinito), imposta `a0 = TERMINATE_SYSCALL` (2, cioè la stessa `SUP_TERMINATE` del Support Level) ed esegue `ecall`: questa è a tutti gli effetti una **SYS2** (`TERMINATE`) che fa terminare la U-proc in modo pulito attraverso `supTerminate`. **Senza questo passo**, il ritorno da `main` (un semplice `ret`) salterebbe a un indirizzo di ritorno indefinito/casuale sullo stack, causando un comportamento imprevedibile o un crash.

La shell (`shell.c`) non sfrutta mai questa via di uscita: il suo `main` contiene un ciclo infinito e termina *solo* tramite una chiamata esplicita a `u_terminate()` (comando `exit`).

7.2 testers/h/libuser.h — la mini libreria utente

```
1 #define TERMINATE      2
2 #define WRITETERMINAL 4
3 #define READTERMINAL  5
4 #define EXECUTE        6
5
6 static void u_print(const char *s) {
7     SYSCALL(WRITETERMINAL, (unsigned int)s, u_strlen(s), 0);
8 }
9 static int u_readline(char *buf) {
10     return (int)SYSCALL(READTERMINAL, (unsigned int)buf, 0, 0);
11 }
12 static int u_execute(int asid) {
13     return (int)SYSCALL(EXECUTE, (unsigned int)asid, 0, 0);
14 }
15 static void u_terminate(void) {
16     SYSCALL(TERMINATE, 0, 0, 0);
17 }
```

È un header *header-only* condiviso da tutti i programmi `testers/`: fornisce wrapper minimi attorno alle quattro syscall del Support Level (`u_print`, `u_readline`, `u_execute`, `u_terminate`), più utility di conversione stringa/intero (`u_strlen`, `u_itoa`, `u_printint`), tutte scritte da zero (non esiste una libc: le U-proc sono “freestanding”, senza runtime C standard).

8 La shell interattiva (testers/shell.c)

```
1 static int nameToAsid(const char *name) {
2     if (u_strcmp(name, "echo") == 0) return 2;
3     if (u_strcmp(name, "fibEleven") == 0) return 3;
4     if (u_strcmp(name, "uname") == 0) return 4;
5     if (u_strcmp(name, "sl") == 0) return 5;
6     if (u_strcmp(name, "calc") == 0) return 6;
7     return -1;
8 }
9
10 void main(void) {
11     char buf[128];
12     u_print("PandOSsh shell\n");
13     u_print("comandi: echo fibEleven uname sl calc | exit\n");
14
15     while (1) {
16         u_print("> ");
17         int n = u_readline(buf);
18         if (n > 0 && buf[n - 1] == '\n') buf[n - 1] = '\0';
19         else buf[n] = '\0';
20
21         if (buf[0] == '\0') continue;
22
23         if (u_strcmp(buf, "exit") == 0) {
24             u_print("shell: arrivederci\n");
25             u_terminate();
26         }
27
28         int asid = nameToAsid(buf);
29         if (asid < 0) { u_print("shell: programma sconosciuto\n"); continue; }
30     }
```

```

31     u_execute(asid);
32 }
33 }

```

La shell (ASID 1, unica U-proc lanciata direttamente dall'InstantiatorProcess) implementa un semplice interprete a riga di comando:

1. Stampa un banner e il prompt "> ".
2. Legge una riga (`u_readline`, bloccante fino a '\n'), rimuove il newline finale sostituendolo con il terminatore di stringa.
3. Riga vuota → richiede subito un nuovo prompt (`continue`).
4. Comando "exit" → stampa un messaggio di commiato e chiama `u_terminate()` (SYS2): non ritorna mai; scatena la catena `supTerminate(1)` → V su `masterSemaphore` → l'InstantiatorProcess si sblocca e porta il sistema a HALT.
5. Altrimenti, traduce il nome del comando in un ASID tramite la mappatura statica `nameToAsid` (ogni programma è precaricato su un proprio device flash dedicato, secondo la corrispondenza ASID-1 = numero del flash: shell→flash0, echo→flash1, fibEleven→flash2, uname→flash3, sl→flash4, calc→flash5). Se il nome non è riconosciuto, avvisa e richiede un nuovo comando; se è valido, chiama `u_execute(asid)` (SYS6), che **blocca la shell** finché il programma lanciato non termina, per poi tornare al prompt.

9 I programmi U-proc lanciabili dalla shell (testers/)

Oltre alla shell (che gira sempre come ASID 1), `testers/` contiene cinque piccoli programmi utente, ciascuno collegato con lo stesso `crtsi.S` e la stessa `libuser.h` viste sopra, e ciascuno precaricato sul proprio device flash dedicato (ASID-1 = numero del flash, coerentemente con `nameToAsid` in `shell.c` e con `flashOperation` in `vmSupport.c`). Sono programmi *applicativi*, non toccano mai direttamente memoria virtuale o Nucleus: usano esclusivamente le quattro syscall wrapper di `libuser.h` (`u_print`, `u_readline`, `u_execute`, `u_terminate`). Il loro scopo è duplice: dimostrare che la paginazione a domanda funziona per programmi realmente eseguiti (ogni istruzione e ogni stringa che leggono risiede in pagine caricate a runtime dal Pager) e coprire i casi richiesti esplicitamente dalla specifica di Phase 3 (una calcolatrice, un echo, ecc.).

9.1 echo.c — ASID 2

```

1  #include "h/libuser.h"
2
3  void main(void) {
4      char buf[128];
5
6      u_print("echo> ");
7      int n = u_readline(buf);
8      if (n < 0) return;
9
10     SYSCALL(WRITETERMINAL, (unsigned int)buf, n, 0);
11 }

```

Il più semplice dei cinque: stampa un prompt, legge una riga (`u_readline`, bloccante fino a '\n', SYS5), e se la lettura non ha restituito un errore (`n < 0`, caso in cui la funzione ritorna subito, terminando il programma tramite il consueto epilogo di `crtsi.S`) la ristampa *esattamente com'è*, incluso il newline finale, con una chiamata **diretta** a `SYSCALL(WRITETERMINAL, ...)` (SYS4) invece di passare per il wrapper `u_print` — equivalente, ma dimostra che `u_print` non è altro che un caso particolare (con `u_strlen` calcolato internamente) della stessa syscall. Verifica quindi in un colpo solo sia `READTERMINAL` sia `WRITETERMINAL`.

9.2 fibEleven.c — ASID 3

```
1 #include "h/libuser.h"
2
3 static int fib(int n) {
4     int a = 0, b = 1;
5     for (int i = 0; i < n; i++) {
6         int t = a + b;
7         a = b;
8         b = t;
9     }
10    return a;
11 }
12
13 void main(void) {
14     u_print("fib(11) = ");
15     u_printint(fib(11));
16     u_print("\n");
17 }
```

Programma puramente computazionale, senza input: calcola l’undicesimo numero della sequenza di Fibonacci con un semplice ciclo iterativo (non ricorsivo: evita di stressare inutilmente lo stack), poi lo stampa con `u_printint` (che internamente usa `u_itoa` per convertirlo in stringa decimale prima di passarlo a `u_print`). Il nome ricorda che in molte varianti storiche di Pandos un programma equivalente calcolava un altro indice della sequenza; qui è fissato a 11. È il candidato ideale per verificare che il **codice** (non solo i dati) di una U-proc venga paginato correttamente: l’intero corpo di `fib` e `main` deve essere caricato dal Pager (pagine `.text`) prima di poter essere eseguito.

9.3 uname.c — ASID 4

```
1 #include "h/libuser.h"
2
3 void main(void) {
4     u_print("PandOSsh - Support Level (Phase 3) - uRISCV\n");
5 }
```

Il programma più minimale possibile: stampa una singola stringa fissa, imitando l’omonimo comando Unix `uname` (che riporta il nome del sistema). Serve soprattutto come “smoke test” per verificare rapidamente che il ciclo `EXECUTE` → `launchUproc` → page fault sul primo accesso a `.text` → esecuzione → `TERMINATE` → `supTerminate` funzioni end-to-end senza alcuna altra variabile in gioco (nessun input, nessun calcolo).

9.4 sl.c — ASID 5

```
1 #include "h/libuser.h"
2
3 void main(void) {
4     u_print("====\n");
5     u_print(" _D _l | |_____/_____ \\__I_I_____==_|_____|\\n");
6     /* ... altre righe di ASCII art ... */
7     u_print("PandOSsh locomotive\n");
8 }
```

Riferimento scherzoso al comando Unix `sl` (uno scherzo storico che “punisce” chi digita `sl` invece di `ls` mostrando un trenino ASCII che attraversa lo schermo): qui è semplicemente una sequenza di `u_print` che stampa più righe di ASCII art. Dal punto di vista tecnico non introduce nulla di nuovo rispetto a `uname.c`, ma essendo un programma più lungo (più stringhe, quindi potenzialmente più pagine `.text` coinvolte) esercita più a fondo il caso in cui una singola esecuzione tocca diverse pagine di codice in sequenza.

9.5 calc.c — ASID 6

```

1  #include "h/libuser.h"
2
3  static void itoa(int v, char *out) {
4      char tmp[16];
5      int i = 0, j = 0, neg = 0;
6      if (v == 0) { out[0] = '0'; out[1] = '\0'; return; }
7      if (v < 0) { neg = 1; v = -v; }
8      while (v > 0) { tmp[i++] = (char)('0' + (v % 10)); v /= 10; }
9      if (neg) out[j++] = '-';
10     while (i > 0) out[j++] = tmp[--i];
11     out[j] = '\0';
12 }
13
14 void main(void) {
15     char buf[128];
16     char out[32];
17
18     u_print("calc: inserisci <cifra><op><cifra> (es. 7*8): ");
19     int n = u_readline(buf);
20
21     if (n < 3) {
22         u_print("calc: input non valido\n");
23         return;
24     }
25
26     int a = buf[0] - '0';
27     char op = buf[1];
28     int b = buf[2] - '0';
29     int r = 0;
30     int ok = 1;
31
32     switch (op) {
33         case '+': r = a + b; break;
34         case '-': r = a - b; break;
35         case '*': r = a * b; break;
36         case '/': if (b == 0) ok = 0; else r = a / b; break;
37         default: ok = 0;
38     }
39
40     if (!ok) {
41         u_print("calc: operazione non valida\n");
42         return;
43     }
44
45     u_print("calc: risultato = ");
46     itoa(r, out);
47     u_print(out);
48     u_print("\n");
49 }

```

Il programma applicativo più completo dei cinque, esplicitamente richiesto dalla specifica della Phase 3: una calcolatrice a riga di comando che accetta espressioni nella forma rigida `<cifra><op><cifra>` (una sola cifra decimale per operando, un solo carattere di operatore — non un vero parser, ma sufficiente per lo scopo dimostrativo).

1. Ha una propria copia locale di `itoa` (identica nella logica a `u_itoa` di `libuser.h`, ma duplicata invece di essere riutilizzata: `libuser.h` è header-only e ogni U-proc è compilata e linkata in modo del tutto indipendente dalle altre, quindi non c'è condivisione automatica di codice applicativo oltre a quanto già offerto da `libuser.h` stessa) usata per convertire il risultato in stringa prima di stamparlo.
2. Legge una riga con `u_readline`; se contiene meno di 3 caratteri (`n < 3`) l'input è per costruzione malformato (non può contenere `<cifra><op><cifra>`) e il programma stampa un errore e ritorna (terminazione pulita via `crt.si.S`, non un program trap: qui l'errore è di logica applicativa, gestito dal programma stesso, non un accesso a memoria invalida).
3. Estrae i tre caratteri (`buf[0]`, `buf[1]`, `buf[2]`), converte le due cifre da carattere ASCII a valore numerico ('0' sottratto), ed esegue l'operazione richiesta con un normale `switch`; la

divisione per zero è esplicitamente controllata (`ok = 0`) invece di lasciarla generare un errore hardware.

4. Se l'operatore non è riconosciuto o la divisione non è valida, stampa un messaggio di errore; altrimenti stampa il risultato convertito in stringa.

Rispetto agli altri quattro programmi, `calc.c` è quello che meglio dimostra l'uso *combinato* di lettura (SYS5) e scrittura (SYS4) sul terminale all'interno di un'unica esecuzione, oltre a un minimo di logica di validazione dell'input — lo stesso principio ("input malformato non deve mai compromettere il sistema") applicato a un livello più grossolano dalle validazioni di `writeTerminal/readTerminal` nel Support Level (Sezione 6).

10 La pipeline di compilazione: da .c a device flash

Esistono due **build indipendenti** nel progetto:

1. Il **kernel** (Nucleus + Support Level): il target CMake `make phase3` compila tutti i moduli C del kernel (compresa la Fase 3) e produce `build/MultiPandOS.core.uriscv` — il file che l'emulatore carica come `core-file`, secondo `phase3_config_machine.json`.
2. I **programmi utente di test** (le U-proc): il Makefile in `testers/` compila ciascun programma in una propria **immagine flash**, indipendente dal core del kernel.

10.1 Il target phase3 in CMakeLists.txt

```
1 add_executable(MultiPandOS_phase3 EXCLUDE_FROM_ALL
2   ./klog.c
3   phase1/pcb.c
4   phase1/as1.c
5   phase2/initial.c
6   phase2/scheduler.c
7   phase2/exceptions.c
8   phase2/interrupts.c
9   phase3/initProc.c
10  phase3/vmSupport.c
11  phase3/sysSupport.c
12  ${URISCV_SRC}/crtso.S
13  ${URISCV_SRC}/liburiscv.S
14 )
15 target_compile_definitions(MultiPandOS_phase3 PRIVATE SUPPORT_LEVEL)
16
17 add_custom_target(phase3
18   COMMAND ${CMAKE_COMMAND} -E copy
19     ${PROJECT_BINARY_DIR}/MultiPandOS_phase3
20     ${PROJECT_BINARY_DIR}/MultiPandOS
21   COMMAND uriscv-elf2uriscv -k ${PROJECT_BINARY_DIR}/MultiPandOS
22   COMMAND ${CMAKE_MAKE_PROGRAM} -C ${CMAKE_SOURCE_DIR}/testers
23   BYPRODUCTS MultiPandOS.core.uriscv MultiPandOS.stab.uriscv
24   DEPENDS MultiPandOS_phase3
25 )
```

Rispetto ai target `phase1/phase2` (documento di Fase 1), qui compaiono *tutti e quattro* i moduli del Nucleus (`phase2/*.c`, *non* `p2test.c`: il processo iniziale `test` viene preso da `phase3/initProc.c`, non più dal test driver) più i tre moduli della Fase 3. Due dettagli decisivi:

- `target_compile_definitions(...PRIVATE SUPPORT_LEVEL)`: è la macro di preprocessore che attiva il ramo `#ifdef SUPPORT_LEVEL` in `uTLB_RefillHandler` (`phase2/exceptions.c` — si veda la Sezione 5.6 di questo documento): *solo* compilando questo target il TLB-Refill handler usa davvero la Page Table della U-proc corrente invece dello skeleton placeholder della Fase 2 pura.
- il target *custom* `phase3` include un **comando aggiuntivo** rispetto a `phase1/phase2`: `${CMAKE_MAKE_PROGRAM} -C ../testers` invoca automaticamente anche il Makefile di `testers/`, così che un singolo `make phase3` produca sia il core del kernel sia le immagini flash di tutte le U-proc, tenendole allineate.

10.2 testers/Makefile — compilazione delle U-proc

```
1 CFLAGS = -ffreestanding -nostdlib -nostartfiles -Wall -O0 -std=gnu99 \  
2         -march=rv32imafd -mabi=ilp32d -I$(URISCV_INC) -Ih  
3 LDFLAGS = -nostdlib -nostartfiles -march=rv32imafd -mabi=ilp32d \  
4         -T $(URISCV_SHARE)/uriscvaout.ldscript  
5  
6 PROGS = shell echo fibEleven uname sl calc  
7 FLASHES = $(PROGS:%=%.uriscv)  
8  
9 all: $(FLASHES)  
10  
11 crtsi.o: crtsi.S  
12     $(CC) $(CFLAGS) -c $< -o $@  
13  
14 liburiscv.o: $(URISCV_SHARE)/liburiscv.S  
15     $(CC) $(CFLAGS) -c $< -o $@  
16  
17 %.o: %.c  
18     $(CC) $(CFLAGS) -c $< -o $@  
19  
20 %.elf: %.o crtsi.o liburiscv.o  
21     $(CC) $(LDFLAGS) crtsi.o $< liburiscv.o -o $@  
22  
23 %.uriscv: %.elf  
24     uriscv-elf2uriscv -a $<  
25     uriscv-mkdev -f $@ $<.aout.uriscv  
26  
27 clean:  
28     rm -f *.o *.elf *.aout.uriscv $(FLASHES)
```

È un Makefile classico basato su **regole implicite con pattern** (`%.o`, `%.elf`, `%.uriscv`): applicando la stessa sequenza di regole a ciascun nome in `PROGS` (`shell echo fibEleven uname sl calc`), produce automaticamente tutte e sei le immagini flash senza dover scrivere una regola separata per ognuna. Per ciascun programma `name.c`, la catena di trasformazioni è:

1. **name.c** → **name.o** (regola pattern generica `%.o: %.c`): compilazione con gli stessi flag “free-standing” del kernel (`-ffreestanding -nostdlib -nostartfiles`, nessuna lib), più `-Ih` per trovare `testers/h/libuser.h`.
2. **Link**: **name.o** + **crtsi.o** + **liburiscv.o** → **name.elf** (regola `%.elf: %.o crtsi.o liburiscv.o`), con lo script linker `uriscvaout.ldscript` (diverso da `uriscvcore.ldscript` usato per il kernel: produce un formato `aout` eseguibile a partire da `0x80000000`, non un’immagine `.core`). **crtsi.o** e **liburiscv.o** sono **oggetti condivisi** da tutti i programmi (compilati una sola volta e riutati nel link di ciascuno): è qui che entra in gioco `crtsi.S`, il cui simbolo `__start` finisce esattamente a `0x800000B0` (`UPROCSTARTADDR`).
3. **Conversione di formato**: `uriscv-elf2uriscv -a` traduce l’ELF nel formato `.aout` (load image) atteso dalla macchina virtuale → **name.elf.aout.uriscv**.
4. **Creazione dell’immagine flash**: `uriscv-mkdev -f name.uriscv name.elf.aout.uriscv` precarica il `.aout` su un device flash virtuale → **name.uriscv** (regola `%.uriscv: %.elf`).

Il target `clean` rimuove tutti gli artefatti intermedi e le immagini finali, per una ricompilazione pulita.

Layout del flash (il backing store di una U-proc): i blocchi 0–30 contengono le pagine di `.text/.data`, il blocco 31 la pagina di stack iniziale — corrispondendo esattamente alle 32 entry della Page Table privata.

10.3 phase3_config_machine.json — la configurazione dell’emulatore

```
1 {  
2     "boot": {  
3         "core-file": "build/MultiPandOS.core.uriscv",  
4         "load-core-file": true  
5     },  
6 }
```

```

6  "devices": {
7      "flash0": { "enabled": true, "file": "testers/shell.uriscv" },
8      "flash1": { "enabled": true, "file": "testers/echo.uriscv" },
9      "flash2": { "enabled": true, "file": "testers/fibEleven.uriscv" },
10     "flash3": { "enabled": true, "file": "testers/uname.uriscv" },
11     "flash4": { "enabled": true, "file": "testers/sl.uriscv" },
12     "flash5": { "enabled": true, "file": "testers/calc.uriscv" },
13     "flash6": { "enabled": false, "file": "" },
14     "flash7": { "enabled": false, "file": "" },
15     "terminal0": { "enabled": true, "file": "term0.uriscv" }
16 },
17 "num-processors": 1,
18 "num-ram-frames": 256,
19 "symbol-table": { "asid": 64, "file": "build/MultiPandOS.stab.uriscv" },
20 "tlb-floor-address": "0x80000000",
21 "tlb-size": 16
22 }

```

Rispetto a `config_machine.json` usato in Fase 1/2 (documento di Fase 1), tre differenze concrete che si ricollegano direttamente a ciò che questo documento ha spiegato:

- **Sei device flash abilitati** (flash0–flash5), ciascuno puntato a un'immagine `testers/*.uriscv` generata dal Makefile appena visto. La **mappatura è posizionale**: il device `flashN` corrisponde all'ASID `N+1`, esattamente come calcolato da `flashOperation` (`devNo = asid - 1`) in `vmSupport.c`. `flash6/flash7` restano disabilitati: `UPROCMAX = 8` ne permetterebbe fino a 8, ma solo 6 programmi sono forniti.
- **num-ram-frames**: 256, molto più dei 64 di Fase 1/2: deve ospitare i 32 frame riservati al Nucleus *più* i 16 frame dello Swap Pool *più* margine per gli stack degli 8 possibili `support_t` (ciascuna con due stack da 500 word) — la memoria virtuale ha bisogno di ben più fisico spazio di manovra del semplice Nucleus.
- **tlb-floor-address**: "0x80000000": a differenza di Fase 1/2 (dove valeva "0x0", nessuno spazio utente da proteggere), qui coincide esattamente con `KUSEG`: dice all'hardware TLB dove inizia lo spazio di indirizzamento *utente* che deve essere tradotto tramite le Page Table private delle U-proc (indirizzi sotto questa soglia restano nello spazio kernel, mappato in modo diretto/identità).

Da tenere bene a mente: il flash *non* è un filesystem generico, ma il backing store grezzo di *una singola* U-proc. Il Pager lo legge a domanda (al verificarsi di un page fault) e vi riscrive le pagine *dirty* sfrattate; non c'è alcuna nozione di file, directory o più programmi condivisi sullo stesso device.

11 Mappa riassuntiva della memoria (Fase 3)

Indirizzo	Costante	Contenuto
0x20000000	RAMSTART	inizio RAM; primi 32 frame riservati al Nucleus
0x20020000	SWAP_POOL_START	inizio dei 16 frame dello Swap Pool
0x80000000	KUSEG / TEXT_VADDR	inizio spazio logico di una U-proc (<code>.text</code>)
0x800000B0	UPROCSTARTADDR	simbolo <code>__start</code> , punto di ingresso reale (dopo l'header <code>.aout</code>)
0xBFFFFFF0	VPN 0xBFFFF	pagina di stack logica (indice 31 della Page Table)
0xC0000000	USERSTACKTOP	cima dello stack utente (valore iniziale di <code>sp</code>)